Speaker Script

Suleyman's Sections

CAS 760 Development Project 2

A Simple File System in Alonzo

Target timing: ∼7 minutes total, ∼60 seconds per slide.

My slides, in order:

1. Files — Theory FILE
2. Secured Files — Theory Extension SECURED-FILE
3. Directories — Theory Extension DIRECTORY
4. Directories as an Inductive Type
5. Directory Development — DIR-DEV
6. Definition of `contains`
7. Definition of `size`

How to read this:

- The *italic indented text* is what I say **verbatim**.
- → **Red arrows** are stage directions — where to point, gesture, or pause.
- *(pause)* means take a breath, about one second.
- ⇒ **Green TRANSITION** means advance to the next slide.
- Each slide starts on a fresh page — flip as you go.

Read this aloud at least twice before the talk. Time yourself.

SLIDE 1 · Files — Theory FILE

timing: ∼70 sec

ON SCREEN: *Base types* N, C, F. *Constants* mkfile, fname, fcontent *with types. Five key axioms listed.*

"Thanks Hamza. Now I'll walk through the Files theory."

→ **Brief nod toward Hamza. Face the audience.**

"The idea here is straightforward: a file is a named container for content. Conceptually, a file is just a pair — a name and some content — but we want to formalize that abstractly, without committing to what names or contents actually are."

"So we introduce three base types —"

→ **Point to the line "N, C, F" on the slide.**

"— N for names, C for content, and F for the file type itself. We keep them opaque on purpose, so that this theory captures the ***structure*** *of files rather than any specific representation."*

(pause)

"We have one constructor, mkfile *—"*

→ **Point to the** mkfile **line.**

"— of type $N \to C \to F$, which builds a file from a name and content. And we have two accessors, fname *and* fcontent *—"*

→ **Point to the** fname **and** fcontent **lines in turn.**

"— that pull the name and content back out."

"The axioms say what you'd expect. mkfile *is total and injective — every name-content pair gives a valid file, and distinct pairs give distinct files. The two accessor axioms say that if you build a file with* mkfile n c*, extracting the name gives you n back, and the content gives*

you c. And the last axiom — surjectivity — says that every file f is equal to mkfile *applied to its own name and content. That closes the loop: F is isomorphic to $N \times C$, but presented abstractly as a theory."*

⇒ **TRANSITION:** advance to Secured Files slide.

SLIDE 2 · Secured Files — Theory Extension SECURED-FILE

timing: ∼75 sec

ON SCREEN: *"Extends: FILE and PERM". "New base types: none (P inherited from PERM)". New constants* perm, protect. *Axioms 8–12.*

> *"SECURED-FILE combines files and permissions. This is one of the more interesting design points in the paper."*

> *"The intuition is: FILE plus PERM equals SECURED-FILE. Each file gets a permission level attached to it. Formally, we present SECURED-FILE as a* ***joint extension*** *of both FILE and PERM —"*

→ **Point to the "Extends: FILE and PERM" line on the slide.**

> *"— this is a pushout-style construction in the little theories method."*

> *"What that means concretely: SECURED-FILE has* ***no new base types*** *of its own."*

→ **Point to "New base types: none" line.**

> *"It inherits P from PERM and inherits N, C, F from FILE. The two extension arrows in the theory graph — from PERM to SECURED-FILE, and from FILE to SECURED-FILE — encode that inheritance formally."*

(pause)

> *"What SECURED-FILE* ***does*** *add is two new constants."*

→ **Point to** perm **line.**

> *"First,* perm*, of type $F \to P$, which tells you the permission of a file."*

→ **Point to** protect **line.**

"And second, protect*, of type $F \to P \to F$, which takes a file and a permission and gives you back the same file with that permission assigned."*

"Axioms 8 through 12 govern these. Eight and nine are totality. Ten is the core semantics: applying protect *with permission p gives a file whose permission is p. And eleven and twelve say that* protect *only changes the permission — it preserves the name and the content. So* protect *is a pure permission-update operation."*

⇒ **TRANSITION:** advance to Directories slide.

SLIDE 3 · Directories — Theory Extension DIRECTORY

timing: ∼70 sec

ON SCREEN: *"Design choice: flat structure." Base type D. Constructors* emptydir, addfile. *Axioms 13–16.*

> *"DIRECTORY is a theory extension over SECURED-FILE. This is where the design choice I want to highlight comes in: we model directories as a* ***flat*** *structure, not as a hierarchical tree."*

> *"The reason is that this keeps the inductive type clean."*

→ **Point to the** addfile **type signature on the slide.**

> *"The constructor* addfile *has type $F \to D \to D$ — it takes a* ***file****, not a directory, as its first argument. That gives us a direct analogy with the list type from the textbook:* emptydir *plays the role of* nil*, and* addfile *plays the role of* cons*."*

> *"If we wanted a hierarchical model, we'd need a second constructor,* adddir *of type $D \to D \to D$ —"*

→ **If whiteboard available: quickly write "`adddir : D → D → D`" in the corner. Otherwise just emphasize with hand gesture.**

> *"— which would make this a tree-shaped inductive type. That's a natural extension, and we mention it as future work in the paper, but for this project the flat model is the right scope."*

(pause)

> *"The axioms are standard. Axiom 13 says* addfile *is total. Axiom 14 says it's* ***injective*** *— and we'll see later that this injectivity is what makes the FILE-to-DIR morphism work."*

→ **Slight emphasis on "injective" — this callback matters for Hamza's morphism section.**

"Axiom 15 is disjointness: addfile *f d is never equal to* emptydir*. And Axiom 16 is the structural induction principle, which lets us prove properties over all directories by handling the two constructor cases."*

⇒ **TRANSITION:** advance to Inductive Type verification slide.

SLIDE 4 · Directories as an Inductive Type

timing: ∼70 sec

ON SCREEN: $K_D = \{\text{emptydir}, (\text{addfile}, U_F)\}$. *Interpretation bullets. Conclusion about finite lists of* D_F^M.

> *"To confirm DIRECTORY is a proper recursive inductive type in Alonzo, we follow the procedure from Chapter 10 of Farmer's textbook."*

> *"The constructor set is* K_D *—"*

→ **Point to the $K_D = \{\ldots\}$ expression on the slide.**

> *"— which contains* emptydir *and the pair* $(\text{addfile}, U_F)$. emptydir *is a* ***first-kind*** *constructor — it's nullary. And* $(\text{addfile}, U_F)$ *is a* ***fourth-kind*** *constructor: the* U_F *means the first argument ranges over the entire domain of* F*, and because* D *appears in the domain of* addfile*'s type, it makes* addfile *recursive."*

(pause)

> *"We check the seven obligations from Chapter 10 in the paper. I'll just mention them briefly: the quasitype for the* F*-argument is non-empty because* F *is a base type; totality and injectivity hold by Axioms 13 and 14;* emptydir *is not in the range of* addfile *by disjointness; disjoint-ranges is trivial since there's only one non-nullary constructor; and structural induction is Axiom 16. All seven pass."*

> *"The consequence, by the Inductive Types Theorem, is that in every standard model* D *is isomorphic to the set of* ***finite lists*** *over the interpretation of* F*. So directories really do behave like lists of files — exactly as the constructor signatures suggested."*

⇒ **TRANSITION:** advance to DIR-DEV intro slide.

SLIDE 5 · Directory Development — DIR-DEV

timing: ∼55 sec

ON SCREEN: *"Derived operations in a development, not as primitive theory constants." Three-bullet list: define via ι-term, validate, prove properties.*

> *"DIR-DEV is a **development** over DIRECTORY, not another theory extension. This distinction matters."*

> *"A development contains **derived** operations — things like size and containment that are definable from the primitives. In Project 1 we made the mistake of putting derived operations into theory extensions as primitive constants, and we lost points for it."*

→ **Slight pause; this sentence earns goodwill by showing we learned from prior feedback.**

> *"The lesson is that derived operations should be **proved to exist**, not postulated."*

> *"So in DIR-DEV we do three things for each operation."*

→ **Point to the three bullets on the slide as you list them.**

> *"First, we define the operation using a **definite description** — that is, an iota-term of the form the unique function satisfying a recursion. Second, we **validate** the definition by proving existence and uniqueness separately. And third, we prove **totality** and any algebraic properties we need as theorems."*

> *"We do this for two operations in the paper:* `contains`*, which checks whether a given name is present in a directory, and* `size`*, which counts the number of files. I'll walk through both."*

⇒ **TRANSITION:** advance to `contains` definition slide.

SLIDE 6 · Definition of `contains`

timing: ∼65 sec

ON SCREEN: $\text{contains}_{N \to D \to \mathbb{B}}$ *as a definite description with two recursion clauses. Two theorems listed at the bottom.*

> *"*`contains` *has type* $N \to D \to \mathbb{B}$*. It takes a name and a directory, and returns true if the directory has a file with that name, false otherwise."*

> *"We define it as the unique function* g *of this type satisfying two recursion clauses."*

→ **Point to the first clause of the definite description.**

> *"First clause: for every name* n*,* $g\ n$ emptydir *equals false — the empty directory contains nothing."*

→ **Point to the second clause.**

> *"Second clause: for every name* n*, file* f*, and directory* d*,* $g\ n$ (addfile $f\ d$) *equals true if* fname f *equals* n*, or if* $g\ n\ d$ *is true. In other words,* addfile $f\ d$ *contains* n *exactly when either* f*'s own name is* n*, or the tail* d *already contains* n*."*

(pause)

> *"The validation in the appendix proves existence by structural induction — the recursion is well-founded because the recursive call is on a strictly smaller directory. Uniqueness is also by structural induction on* D*."*

> *"Once validated, we get two theorems for free —"*

→ **Point to the theorems at the bottom of the slide.**

> *"—* `contains` *is total, and* `contains` n emptydir *is false for every* n*."*

⇒ **TRANSITION:** advance to `size` definition slide.

SLIDE 7 · Definition of `size`

timing: ∼70 sec

ON SCREEN: $\text{size}_{D \to \mathbb{N}}$ *as a definite description with two recursion clauses. Four theorems listed below.*

> *"*`size` *has type* $D \to \mathbb{N}$*. It counts the number of files in a directory."*

> *"Again, we define it as the unique function* g *satisfying two recursion clauses."*

→ **Point to the first clause.**

> *"*g emptydir *equals zero."*

→ **Point to the second clause.**

> *"And for every file* f *and directory* d*,* g (addfile f d) *equals* g d *plus one. So adding a file to a directory increases the size by exactly one."*

> *"Validation is the same pattern: existence by structural induction, uniqueness by structural induction."*

(pause)

> *"We prove four theorems about* `size` *—"*

→ **Point to each theorem in order as you list them.**

> *"First, totality. Second,* `size` emptydir *equals zero. Third, the recurrence:* `size` (addfile f d) *equals* `size` d *plus one. And fourth, non-negativity:* `size` d *is at least zero for every* d *— this one is by structural induction using the recurrence."*

> *"That completes my sections of the development. Hamza will now take over with the theory morphism and the conclusion."*

⇒ **HAND OFF:** look to Hamza, step slightly aside, smile.

CONTINGENCY · Example of Directory Evaluation

only if I'm asked to present the Example slide instead of Hamza

ON SCREEN: *Two files f_1, f_2 defined via* protect. *Two-entry directory d. Values of* `contains` *and* `size` *at the bottom.*

"A concrete example. Let f_1 and f_2 be two secured files —"

→ **Point to f_1 and f_2 definitions on the slide.**

*"— f_1 has name n_1, content c_1, and **read** permission; f_2 has name n_2, content c_2, and **write** permission. Let d be the two-entry directory* addfile f_1 (addfile f_2 emptydir)*."*

"Unfolding `contains` *—"*

→ **Point to each contains equation on the slide.**

"— `contains` *n_1 d is true because f_1's name matches at the outer layer.* `contains` *n_2 d is also true — we recurse into the tail and find f_2.* `contains` *n_3 d is false — no match at either layer."*

"And unfolding `size`*:* `size` *d equals* `size` *of the one-entry tail plus one, which equals* `size` *of* emptydir *plus two, which is two. So this two-entry directory has size two, which confirms the theorems."*

Q&A PREP · Likely Questions

rehearse these — one of them almost certainly comes up

Q1. Why flat directories and not hierarchical?

"Scope — the project name says ***simple*** *file system. Flat keeps the inductive type within the standard fourth-kind constructor pattern of Chapter 10, preserves the direct list analogy, and makes the FILE-to-DIR morphism clean because single-entry directories give a canonical target for the embedding. Hierarchy would require a second recursive constructor* $\mathrm{adddir} : D \to D \to D$*, which breaks the clean list analogy and makes the morphism extraction ambiguous. We note hierarchy as future work in the paper."*

Q2. Why are `contains` and `size` in a development instead of extending DIRECTORY?

"Because they're ***derived*** *operations — they're definable from the primitives* addfile *and* emptydir*, so postulating them as theory constants would add redundant axioms. The development framework lets us define them once, prove they exist uniquely, and then use them. In Project 1 we put derived operations into theory extensions and lost points for it — this is the direct correction."*

Q3. How did you verify the DIRECTORY obligations?

"All seven obligations from Chapter 10 of Farmer's textbook — non-empty quasitypes, totality on domain, injectivity, disjointness of ranges, structural induction. Each one discharges directly from one of our four axioms, Axioms 13 through 16, with the non-emptiness obligation for the F*-argument holding because* F *is a base type and every base type is inhabited in any Alonzo frame. The full check is in the paper."*

Q4. What if two files in a directory have the same name?

"They're allowed. The inductive type doesn't enforce uniqueness — you can have two addfile *applications with files sharing a name.* `contains` *would just return true if* ***any*** *match is found. A future extension could*

add a uniqueness invariant, but that's a development-layer property, not a theory-layer one."

Q5. Why use K_D for the constructor set?

"Standard inductive-type convention — K for Konstruktor, subscripted by the type. The subscript makes it unambiguous which type's constructors we mean, and it avoids clashing with the base type C of FILE."

Q6. What does "pushout-style construction" mean?

"It's a way of combining two theories that share no non-trivial subtheory. SECURED-FILE takes FILE and PERM as inputs and produces a new theory whose language is the union of both, augmented with the new constants perm *and* protect*. The category-theoretic term is a pushout over the empty theory. In Alonzo's little theories method, we encode it by having SECURED-FILE extend both parents simultaneously, which is what the two arrows in our theory graph represent."*